

Compression d'un signal ECG par DCT unidimensionnelle

Sep 29, 2026 · @Slava

Projet de fin d'études du DUT Réseaux et Télécommunications (IUT de Vélizy, soutenance du 31 août 2022), repris et finalisé en 2026.

1. Introduction

Ce projet compresse un ECG de la base PTB-XL par DCT, produit un vrai fichier compressé, puis reconstruit le signal au format d'origine. Il répond à la consigne : « Compresser le signal ECG en appliquant une DCT unidimensionnelle et en mettant à zéro les coefficients avec la plus faible énergie ».

Un ECG génère beaucoup de données. Une seule dérivation à 250 échantillons par seconde, codée sur 12 bits, représente environ 32 Mo par jour. Un service de cardiologie enregistre plusieurs dérivations pour de nombreux patients, souvent pendant des heures.

La compression garde aujourd'hui tout son intérêt pour les appareils portables (holters, montres, patchs cardiaques). Leur bande passante et leur batterie sont limitées, et chaque octet transmis consomme de l'énergie.

Les objectifs du projet sont les suivants :

1. Calculer la DCT d'un signal ECG réel.
2. Mettre à zéro les coefficients de plus faible énergie.
3. Enregistrer le résultat dans un fichier compressé et comparer sa taille à l'original.
4. Reconstruire l'ECG, mesurer la perte de qualité et comparer à nouveau les tailles.

2. Le signal ECG

L'électrocardiogramme enregistre l'activité électrique du cœur à l'aide d'électrodes posées sur la peau. À chaque battement, une impulsion part du nœud sinusal et se propage dans le muscle cardiaque, qui se contracte.

Chaque battement produit trois éléments caractéristiques sur le tracé :

Élément	Événement cardiaque	Forme sur le tracé
Onde P	Contraction des oreillettes	Petite bosse arrondie
Complexe QRS	Contraction des ventricules	Pic bref et intense (pic R)

Élément	Événement cardiaque	Forme sur le tracé
Onde T	Relâchement des ventricules	Bosse plus large

Un ECG standard comporte 12 dérivations (I, II, III, aVR, aVL, aVF, V1 à V6). Chacune observe la même activité électrique sous un angle différent.

Pour la compression, une propriété compte particulièrement : l'essentiel de l'information utile d'un ECG se situe en dessous de 40 Hz environ. Au-delà, le signal contient surtout du bruit (activité musculaire, parasites électriques). Cette concentration dans les basses fréquences est ce qui rend la DCT efficace.

3. Les données : PTB-XL et le format WFDB

Le projet utilise PTB-XL, une base publique hébergée par PhysioNet. La version 1.0.1 contient 21 837 ECG cliniques de 10 secondes, à 12 dérivations, provenant de 18 885 patients.

Chaque ECG existe en deux résolutions :

Version	Suffixe	Dossier	Fréquence	Échantillons par dérivation
Basse résolution	<code>_1r</code>	<code>records100/</code>	100 Hz	1 000
Haute résolution	<code>_hr</code>	<code>records500/</code>	500 Hz	5 000

Un enregistrement est stocké au format WFDB, sous forme de deux fichiers :

- `.hea` : un en-tête texte (fréquence, noms des dérivations, unités, gain).
- `.dat` : les échantillons, stockés en entiers de 16 bits (2 octets), avec une résolution de 0,001 mV.

La bibliothèque Python `wfdb` lit ces fichiers et renvoie les signaux en millivolts. Les essais de ce rapport portent sur l'enregistrement `00001_1r`, dont le fichier `.dat` pèse $12 \times 1\,000 \times 2 = 24\,000$ octets.

4. La transformée en cosinus discrète (DCT)

La DCT réécrit un signal de N échantillons comme une somme de N cosinus de fréquences croissantes. Chaque coefficient indique la part d'un cosinus dans le signal : on passe du domaine temporel au domaine fréquentiel.

DCT-II orthonormée (transformée directe)

$$X_k = \sqrt{\frac{2}{N}} \sum_{n=0}^{N-1} x_n \cos\left[\frac{\pi}{2N}(2n+1)k\right], \quad \alpha_0 = \frac{1}{\sqrt{2}}, \quad \alpha_{k>0} = 1$$

DCT-III orthonormée (transformée inverse)

$$x_n = \sqrt{\frac{2}{N}} \sum_{k=0}^{N-1} \alpha_k X_k \cos\left[\frac{\pi}{2N}(2n+1)k\right]$$

Le coefficient d'indice k correspond à la fréquence $k \times f_s / (2N)$. Pour 00001_1r ($f_s = 100$ Hz, $N = 1000$), l'indice 0 représente la moyenne du signal et l'indice 999 la fréquence maximale, 50 Hz.

Pourquoi la DCT compresse

La DCT regroupe l'énergie d'un signal régulier dans un petit nombre de coefficients, principalement en basses fréquences. Les autres coefficients sont proches de zéro et peuvent être supprimés avec peu de pertes. Les formats JPEG, MPEG et, sous une forme modifiée, MP3 et AAC reposent sur ce principe.

Énergie et théorème de Parseval

L'énergie d'un coefficient est son carré. Avec la normalisation orthonormée, l'énergie totale des coefficients est égale à celle du signal :

$$\sum_{n=0}^{N-1} x_n^2 = \sum_{k=0}^{N-1} X_k^2$$

Conséquence directe : l'énergie de l'erreur de reconstruction est exactement l'énergie des coefficients supprimés. Si E est la fraction d'énergie conservée, l'erreur relative (PRD, définie en section 7) vaut :

$$\mathrm{PRD} = 100 \times \sqrt{1 - E}$$

Conserver 99,9 % de l'énergie donne donc un PRD théorique de 3,16 %, quel que soit l'ECG.

5. Méthode de compression

La compression suit quatre étapes, appliquées séparément à chacune des 12 dérivations. La décompression les inverse.

Compression

1. **DCT** : calcul des N coefficients de la dérivation.
2. **Mise à zéro** : les coefficients sont triés par énergie décroissante. On conserve les plus forts jusqu'à atteindre l'énergie cible (99,9 % par défaut) ; tous les autres sont mis à zéro.
3. **Quantification** : les coefficients conservés sont arrondis au pas de 0,001 mV (la résolution du fichier d'origine), puis stockés en entiers de 16 bits (32 bits si les valeurs dépassent cette plage).
4. **Compression sans perte** : le tableau d'entiers est écrit avec `np.savez_compressed` (algorithme deflate, celui du format zip). Les longues suites de zéros n'occupent alors presque plus de place.

Pourquoi la quantification est indispensable

Mettre des coefficients à zéro ne réduit pas, à lui seul, la taille du fichier. Le `.dat` d'origine code chaque échantillon sur 2 octets, alors qu'un coefficient DCT calculé en Python est un nombre à virgule de 8 octets. Sans quantification, le fichier « compressé » serait jusqu'à quatre fois plus gros que l'original. Cette chaîne DCT, quantification et codage sans perte est celle du format JPEG.

Décompression

1. Lecture du fichier compressé et conversion des entiers en millivolts (multiplication par le pas).
2. DCT inverse de chaque dérivation.
3. Écriture de l'ECG reconstruit au format WFDB (`.hea` et `.dat`), avec le même gain et le même codage 16 bits que l'original. Le résultat se lit avec `wfdb` comme n'importe quel ECG de PTB-XL.

La reconstruction redonne un fichier de même taille que l'original : la compression sert au stockage et à la transmission, et l'on ne décompresse que pour relire le signal.

6. Implémentation

Le programme est découpé en trois fichiers Python, sur le modèle d'un outil de compression classique : un script pour compresser, un pour décompresser, et un module de fonctions partagées.

Fichier	Rôle	Entrée	Sortie
<code>compresser.py</code>	Compression et comparaison des tailles	ECG d'origine (<code>.hea</code> + <code>.dat</code>)	Fichier <code>.npz</code> + graphiques
<code>decompresser.py</code>	Reconstruction, mesure de l'erreur, comparaison des tailles	Fichier <code>.npz</code>	ECG reconstruit (<code>.hea</code> + <code>.dat</code>) + graphiques
<code>ecg_dct.py</code>	DCT, DCT inverse, mise à zéro, mesures (importé par les deux scripts)	—	—

La DCT et la DCT inverse sont codées « à la main », en double boucle, d'après les formules de la section 4. Une option `--rapide` utilise à la place `scipy.fft` : le résultat est identique (écart inférieur à 10^{-13}), mais le calcul est environ mille fois plus rapide, ce qui est utile pour les ECG à 500 Hz.

Environnement

Le projet fonctionne en local, sans connexion internet une fois installé. Sous Debian ou Ubuntu récents, les bibliothèques s'installent dans un environnement virtuel :

```
sudo apt install python3-venv
python3 -m venv venv
source venv/bin/activate
pip install wfdb numpy scipy matplotlib
```

Utilisation

```
python compresser.py 00001_lr
python decompresser.py sortie/compresse/00001_lr.npz
```

Option	Script	Effet	Défaut
<code>--energie</code>	compresser	Fraction d'énergie conservée	0.999
<code>--pas</code>	compresser	Pas de quantification (mV)	0.001
<code>--original</code>	decompresser	Chemin de l'ECG d'origine, pour la comparaison	dossier courant

Option	Script	Effet	Défaut
<code>--rapide</code>	les deux	DCT rapide (scipy)	boucle manuelle
<code>--sortie</code>	les deux	Dossier de sortie	<code>sortie</code>

Fichiers produits

```

sortie/
├─ compresse/00001_lr.npz
├─ reconstruit/00001_lr_reconstruit.hea / .dat
└─ graphiques/00001_lr/
    ├─ compression/      signal d'origine + coefficients DCT
    └─ reconstruction/   original et reconstruit superposés + erreur

```

Chaque graphique indique ses unités (temps en secondes, amplitude en mV). Le graphique des coefficients porte un double axe : l'indice k en bas et la fréquence correspondante en hertz en haut ; les coefficients conservés y apparaissent en noir, les coefficients supprimés en rouge.

7. Résultats

Sur la dérivation II de `00001_lr`, conserver 99,9 % de l'énergie garde 664 coefficients sur 1 000, avec un PRD de 3,16 %, exactement la valeur prévue par Parseval. Le signal reconstruit se superpose au signal d'origine, et l'erreur ressemble à un bruit aléatoire d'environ $\pm 0,008$ mV, sans forme d'onde reconnaissable.

Indicateurs utilisés

Indicateur	Définition	Lecture
CR (taux de compression des coefficients)	N / nombre de coefficients conservés	2 = un coefficient sur deux gardé
PRD (%)	$100 \times \sqrt{\text{énergie de l'erreur} / \text{énergie du signal}}$	< 2 % très bon, 2 à 9 % bon, > 9 % déformation visible
Gain en taille	Taille d'origine / taille du fichier	Mesuré sur les fichiers réellement écrits

Qualité selon l'énergie conservée

Énergie conservée	PRD théorique	Qualité attendue
99,9 %	3,16 %	Très bonne
99,5 %	7,07 %	Bonne
99 %	10 %	Limite : vérifier les ondes P

Après reconstruction, le PRD mesuré dépasse très légèrement la valeur théorique (de l'ordre de 0,01 à 0,03 point) à cause de l'arrondi des coefficients.

Tailles de fichiers (00001_1r , 99,9 %)

Fichier	Taille (octets)	Part de l'original
ECG d'origine (.hea + .dat)	24 000 + taille du .hea	100 %
Référence : origine compressée sans perte (zip)	à compléter	à compléter
ECG compressé par DCT (.npz)	à compléter	à compléter
ECG reconstruit (.hea + .dat)	à compléter	environ 100 %

- ☐ Reporter les tailles affichées par `compresser.py` et `decompresser.py` sur 00001_1r .
- ☐ Ajouter les deux graphiques de la dérivation II (compression et reconstruction).

Le CR de 1,51 est modeste. Deux raisons l'expliquent : le signal contient un bruit réparti sur toutes les fréquences, que le critère de 99,9 % oblige à conserver ; et à 100 Hz, l'ECG occupe déjà une grande partie de la bande disponible (0 à 50 Hz). La version à 500 Hz, dont les coefficients au-delà de 50 Hz sont presque tous négligeables, devrait se compresser nettement mieux.

8. Discussion et limites

La méthode fonctionne et reste fidèle au signal, mais elle n'est pas la plus performante pour un ECG. Trois limites sont à connaître.

La DCT n'est pas localisée dans le temps. Chaque coefficient décrit un cosinus sur les 10 secondes entières. Or un ECG est fait d'événements brefs (les complexes QRS) séparés de zones calmes. L'énergie de chaque pic se disperse alors sur de nombreux coefficients.

Les ondelettes (DWT), localisées à la fois en temps et en fréquence, sont pour cette raison l'approche la plus répandue en compression ECG. La DCT par blocs, comme les blocs 8×8 du JPEG, est un compromis intermédiaire.

Le PRD ne garantit pas la valeur diagnostique. Un PRD global faible peut masquer une erreur concentrée sur une petite onde P ou sur le segment ST, deux zones importantes pour le diagnostic. Une compression avec perte de données médicales doit être validée cliniquement avant tout usage réel.

Le critère d'énergie conserve le bruit. Viser 99,9 % de l'énergie oblige à garder une partie du bruit de haute fréquence, qui n'a aucune valeur médicale. Un filtrage préalable, ou un critère fondé directement sur un PRD cible, améliorerait le taux de compression.

9. Retour sur la version de 2022

La version de 2022 échouait à cause d'une seule erreur dans la boucle de la DCT ; la formule et sa normalisation étaient correctes.

```
for i in range(0, nSamples):
    for n in range(0, nSamples):
        signal = record.p_signal[n, iSignal]
        sum = signal * math.cos((math.pi*(2*n+1)*i) / (2*nSamples))
```

L'instruction `sum = ...` remplaçait la valeur à chaque tour au lieu de l'additionner. Chaque coefficient ne contenait donc que le dernier terme de la somme. Ce terme se simplifie en $(-1)^i \times \cos(\pi / 2N) \times x[N-1] \times \sqrt{2/N}$: un signe qui alterne à chaque coefficient, sous une enveloppe qui décroît vers zéro. C'est exactement la zone rouge « en trompette » obtenue en 2022, dont l'amplitude (environ 0,001 mV pour la dérivation I) correspond à la dernière valeur du signal multipliée par $\sqrt{2/1000}$.

Problème de 2022	Correction
<code>sum = ...</code> écrase la valeur	<code>sum += ...</code>
<code>sum</code> jamais remis à zéro	<code>sum = 0</code> avant chaque coefficient
Coefficients tracés en fonction du temps	Tracés en fonction de l'indice k, avec l'axe des fréquences
Pas de mise à zéro ni de DCT inverse	Étapes ajoutées (sections 5 et 6)
Aucun fichier en sortie	Fichier compressé <code>.npz</code> et ECG reconstruit <code>.hea / .dat</code>
Google Colab, en ligne	Scripts locaux, fonctionnement hors ligne

Une fois corrigée, la double boucle donne les mêmes coefficients que `scipy.fft.dct` en mode orthonormé, à 5×10^{-14} près. Elle est conservée dans le module `ecg_dct.py` comme méthode de calcul par défaut.

10. Conclusion et perspectives

Le projet répond à la consigne : l'ECG est compressé par DCT unidimensionnelle en supprimant les coefficients de plus faible énergie, enregistré dans un fichier plus léger, puis reconstruit au format d'origine avec une erreur maîtrisée (PRD de 3,16 % pour 99,9 % de l'énergie). La relation $PRD = 100 \times \sqrt{1 - E}$, issue du théorème de Parseval, permet de choisir le niveau de compression à l'avance en fonction de la qualité voulue.

Plusieurs pistes permettraient d'aller plus loin :

- Appliquer la DCT par blocs et mesurer le gain par rapport à la DCT globale.
- Comparer avec une compression par ondelettes (bibliothèque `pywavelets`).
- Tester plusieurs ECG, dont des tracés pathologiques, pour voir si le taux de compression dépend du patient.
- Remplacer le critère d'énergie par un PRD cible, plus proche des pratiques en compression ECG.
- Traiter automatiquement un lot d'enregistrements et produire un tableau de résultats.

Références

- [PTB-XL, a large publicly available electrocardiography dataset, version 1.0.1 \(PhysioNet\)](#)
- [Waveform Database Software Package \(WFDB\) for Python, version 3.3.0 \(PhysioNet\)](#)